

Swapnil Prakash Lader

AI Engineer — Backend & MLOps

claude1128@crowdanalytix.com · github.com/swapnillader · linkedin.com/in/swapnil-mlops-eng · medium.com/@swapnillader21 · swapnillader.github.io/resume

SUMMARY

Backend and MLOps engineer with 5 years of experience shipping LLM systems to production — scalable APIs, CI/CD automation, and self-healing infrastructure. I build AI-powered systems with a focus on scale, structure, and long-term reliability: clean architecture, readable code, and infrastructure that keeps making sense as systems grow. I design for failure as a first-class concern — fallbacks, retries, and adaptive workflows that keep systems reliable under real-world conditions.

EXPERIENCE HIGHLIGHTS

- Five years across backend engineering, scalable APIs, and MLOps/DevOps for production AI systems.
- Built batch pipelines processing millions of SKUs with idempotent retries and dead-letter handling.
- Ran an autoscaling chatbot backend serving hundreds of thousands of users.
- Brought production-grade CI/CD deployment culture to on-prem LLM workstations using vLLM and Jenkins.

SELECTED PROJECTS

On-Prem LLM Deployment Pipeline — Python · vLLM · Jenkins · CI/CD

- Automated CI/CD pipeline treating on-prem workstations like a production fleet: versioned model rollouts served through vLLM, health checks, and clean rollback paths.
- Cut deployment time from ad-hoc manual work to a predictable, orchestrated process and established a production-grade deployment culture across the org.

SKU Batch Processing Platform — Python · FastAPI · AWS · Grafana

- Designed end-to-end batch architecture for millions of SKUs: chunked processing with idempotent retries, dead-letter handling for poison records, and Grafana dashboards for throughput and failure rates.
- Pipeline self-heals from partial failures instead of requiring manual restarts.

Autoscaling Chatbot Platform — Python · Google Cloud · Autoscaling

- Built chatbot backend serving hundreds of thousands of users on autoscaling infrastructure: load-aware scaling policies, graceful connection draining, and fallback responses when downstream models are saturated.
- System stays responsive under real-world load patterns, scaling out under load and back when idle without dropping conversations.

SKILLS

Python

FastAPI

Flask

Django

vLLM

Jenkins

Grafana

AWS

Google Cloud

CI/CD